

Ingeniería en Computación

ELECTRÓNICA DIGITAL

Teórico 1

Sistemas de Representación Numérica y Aritmética Binaria



Universidad Nacional
de **Río Negro**

Magnitudes vs. Números

- Lo que conocemos como **números** son formas de expresar **magnitudes** que son preexistentes
 - Por ejemplo: los dedos de las manos de un ser humano siempre fueron diez, aun antes de que existiera el número 10, X, etc.
- Usar el **sistema decimal** condiciona la forma en la que comprendemos las magnitudes pues es la forma en la que las racionalizamos, por ejemplo, en la escritura.
 - Las culturas mesopotámicas usaban el sistema duodecimal (base 12), introducido por los astrónomos (12 meses solares, 12 horas de día, 12 horas de noche, etc.) acabó por regir el comercio a través de la docena, etc.

Sistemas de Numeración (1/2)

- Un **sistema de numeración** es un **conjunto de símbolos** y un **conjunto de reglas** que permiten combinarlos de forma que es posible representar información, en particular, magnitudes
- Entre los sistemas de numeración, una clase de suma importancia son los **sistemas posicionales**

$$a_n \dots a_2 a_1 a_0 = a_n \cdot b^n + \dots + a_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0$$

Sistemas de Numeración ^(2/2)

- **Sistema decimal:** base = 10. Es el sistema de numeración de uso cotidiano.

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- **Sistema binario:** base = 2. Es el sistema de numeración sobre el que se basa la construcción de componentes de electrónica digital.

$\{0, 1\}$

- **Sistema octal:** base = 8. Se utiliza para simplificar la representación de binarios largos. También en los permisos de Linux.

$\{0, 1, 2, 3, 4, 5, 6, 7\}$

- **Sistema hexadecimal:** base = 16. Muy utilizado en la representación compacta tanto del contenido de la memoria de una computadora, como de las direcciones de sus palabras, pues 2 dígitos hexadecimales son *1 byte*.

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$

Cambio de Base – Conversión a Decimal

- Una representación de una magnitud en un sistema de numeración **puede ser modificada** a partir de cambiar la base, preservando la magnitud.
- Si observamos que para nosotros las magnitudes son racionalizadas en base 10.
 - De magnitud a representación:

$$(a_n \dots a_2 a_1 a_0)_{10} = a_n \cdot 10^n + \dots + a_2 \cdot 10^2 + a_1 \cdot 10^1 + a_0$$

$$(b_n \dots b_2 b_1 b_0)_c = b_p \cdot c^p + \dots + b_2 \cdot c^2 + b_1 \cdot c^1 + b_0$$

- Ejemplos:

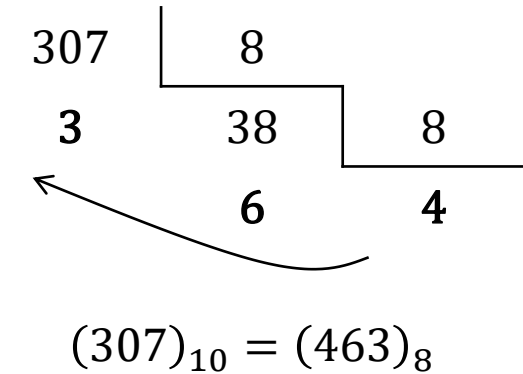
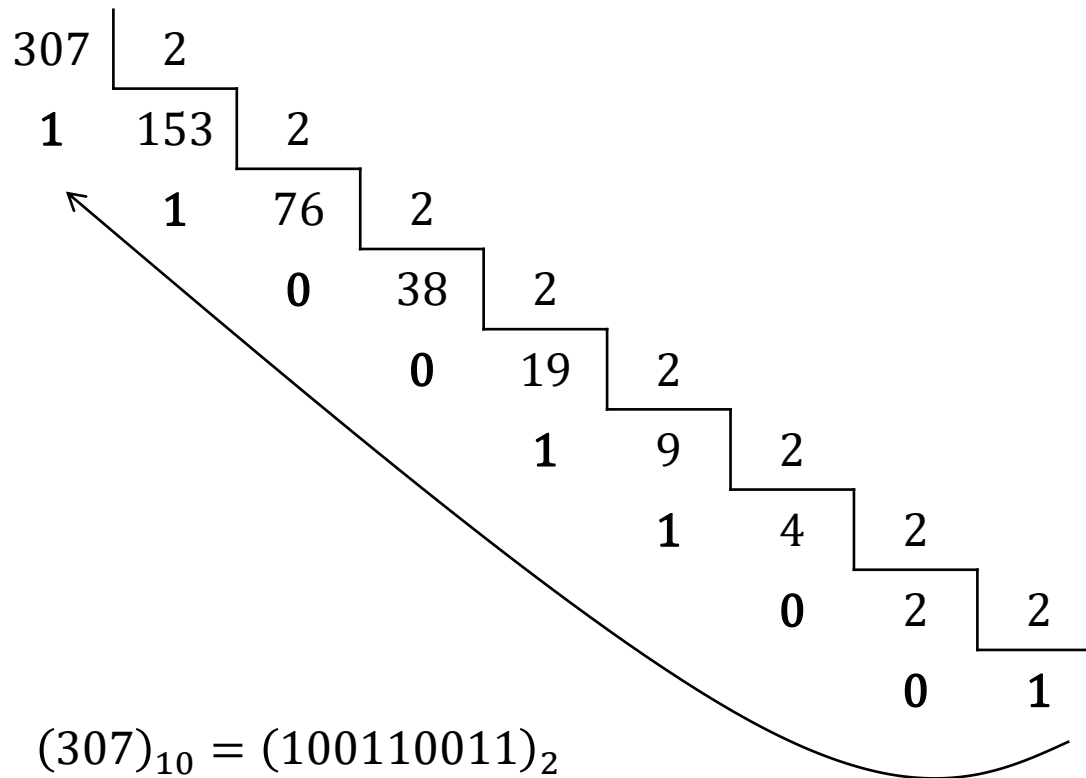
$$(307)_{10} = 3 \cdot 10^2 + 0 \cdot 10^1 + 7$$

$$(463)_8 = 4 \cdot 8^2 + 6 \cdot 8^1 + 3$$

$$(100110011)_2 = 1 \cdot 2^8 + 0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1$$

Conversión Sistema Decimal a Otra base

■ Restos de Cocientes



Sistema Hexadecimal

Decimal	Binario	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

- Sistema en base 16.
- Se usa porque 1 dígito Hexa = 4 bits.
- Se los expresa con el prefijo *0x*

Conversión: Binario a Hexadecimal

- Binario a Hexadecimal: agrupar los dígitos de a 4 bits

(a) $\begin{array}{cccc} 1100 & 1010 & 0101 & 0111 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ C & A & 5 & 7 \end{array} = \mathbf{CA57}_{16}$

(b) $\begin{array}{ccccc} 0011 & 1111 & 1000 & 1011 & 0100 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 3 & F & 1 & 6 & 9 \end{array} = \mathbf{3F169}_{16}$

- Hexadecimal a Binario: cada dígito hexa se expande a 4 bits

(a) $\begin{array}{cccc} 1 & 0 & A & 4 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 0000 & 1010 & 0100 \end{array}$

(b) $\begin{array}{cccc} C & F & 8 & E \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 1100 & 1111 & 1000 & 1110 \end{array}$

Conversión: Hexadecimal a Decimal

- “Igual” que en binario, salvo que multiplicas cada dígito por la potencia de 16 correspondiente.

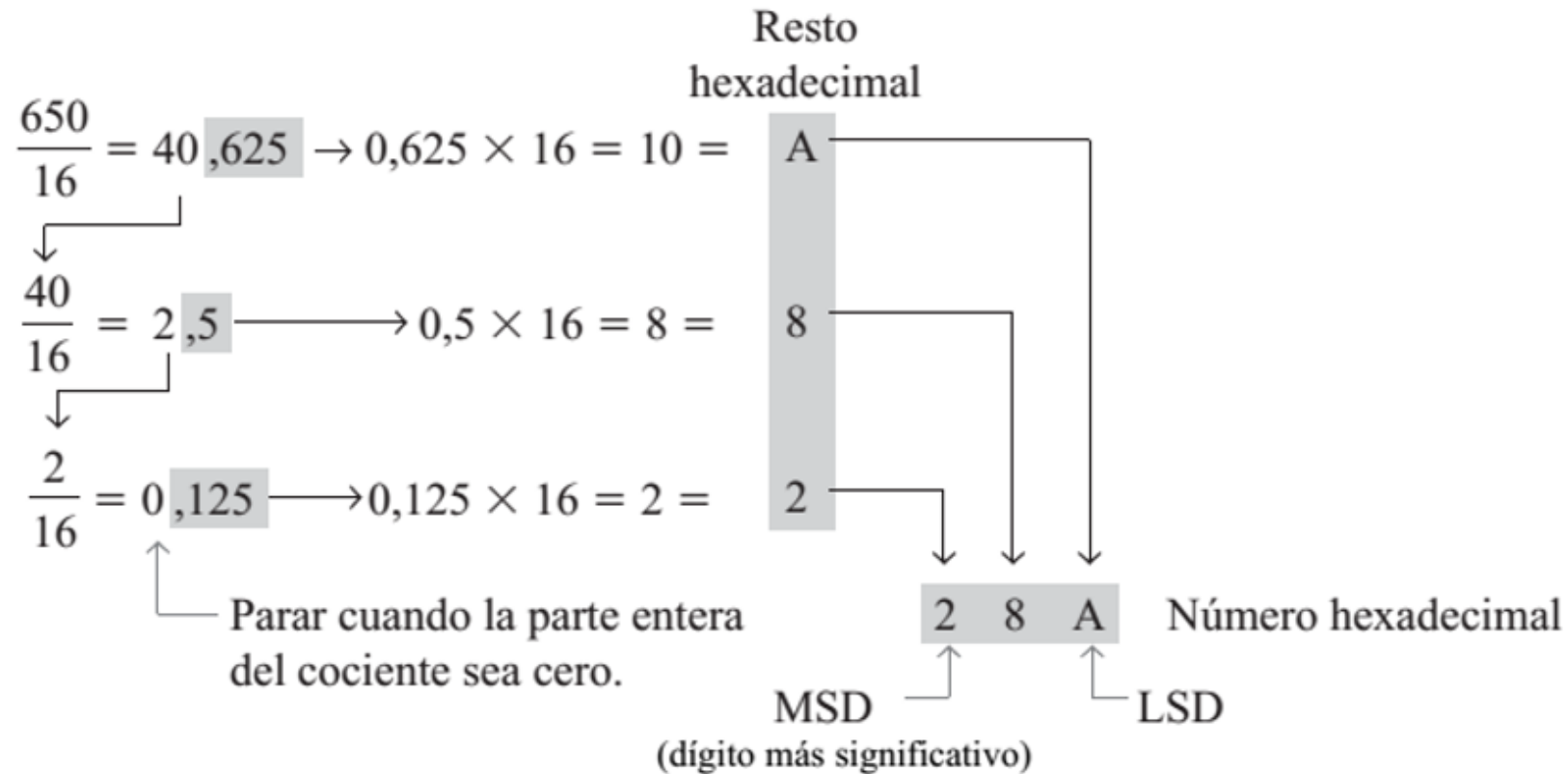
$$\begin{aligned} A85_{16} &= A \times 16^2 + 8 \times 16^1 + 5 \times 16^0 = 10 \times 256 + 8 \times 16 + 5 \\ &= 2560 + 128 + 5 = 2693 \end{aligned}$$

- ... o llevas a binario y de ahí a decimal

$$\begin{array}{ccc} A & 8 & 5 \\ \downarrow & \downarrow & \downarrow \\ \underbrace{1010}_{A} & \underbrace{1000}_{8} & \underbrace{0101}_{5} \end{array} = 2^{11} + 2^9 + 2^7 + 2^2 + 2^0 = 2048 + 512 + 128 + 4 + 1 = \mathbf{2693}_{10}$$

Conversión: Decimal a Hexadecimal

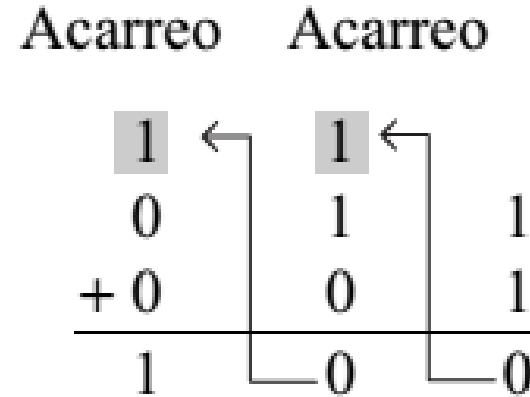
- “Igual” que en binario: división sucesiva por 16



Aritmética Binaria – sin signo

■ Suma

$0 + 0 = 0$	Suma 0 con acarreo 0
$0 + 1 = 1$	Suma 1 con acarreo 0
$1 + 0 = 1$	Suma 1 con acarreo 0
$1 + 1 = 10$	Suma 0 con acarreo 1



■ Resta

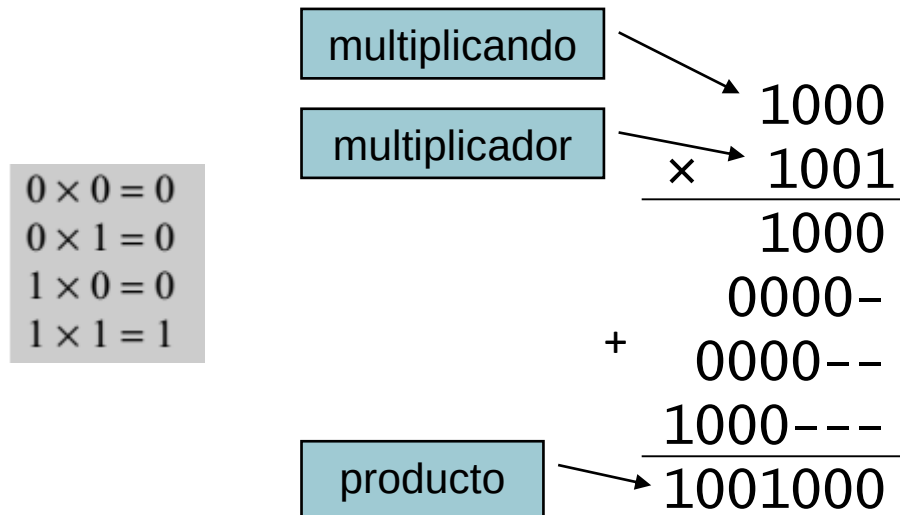
$0 - 0 = 0$	
$1 - 1 = 0$	
$1 - 0 = 1$	
$10 - 1 = 1$	0 - 1 con acarreo negativo de 1

101	5
<u>-011</u>	<u>-3</u>
010	2

*En la bibliografía en *inglés*: Acarreo: **Carry** | Acarreo negativo (o préstamo): **Borrow**

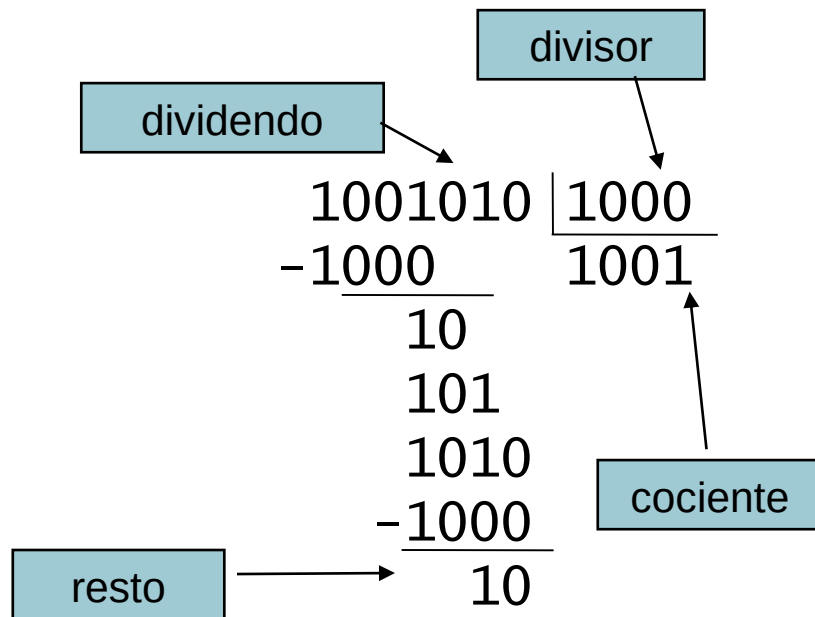
Multiplicación – sin signo

- Como nos enseñaron en la escuela



- Para binaries sin signo, cada paso de la multiplicación es muy simple:
 - Copias el multiplicando en el lugar correcto si el dígito del multiplicador es 1.
 - Copias 0 si el dígito es 0.

División – sin signo



*Operandos de n -bits dan cociente y resto de n -bits

- Verificar que el divisor no sea 0
- Enfoque de la escuela
 - Si el divisor $\leq$ dividendo
 - Bit 1 en cociente, restar
 - Si no
 - Bit 0 en cociente, bajar el próximo bit del dividendo

Magnitudes Signadas: Números Enteros Binarios

- Existen distintas formas de representar a los números con signo:
 - **Signo y Magnitud**
 - **Sistema de Complemento**
 - Complemento a 1
 - Complemento a 2

Representación Signo y Magnitud

- Se utiliza el dígito más a la izquierda para representar el signo
 - El número $(a_n a_{n-1} \dots a_1 a_0)_b$ es positivo si $a_n = 0$ y negativo si $a_n = 1$.
 - $a_{n-1} \dots a_1 a_0$ representa la magnitud.
 - Permite representar las magnitudes $[-(b^{k-1} - 1), b^{k-1} - 1]$.
- **Problemas de esta representación:**
 - No es obvio dónde va el bit de signo.
 - Los sumadores necesitan un paso extra para determinar el signo del resultado (y determinar la operación correspondiente)
 - Existen dos ceros (1000 y 0000).

00011001
Bit de signo ↑ ↑ Bits de magnitud

El número decimal -25 se expresa como:

10011001

Representación Complemento a 1

- Se utilizan los números “más grandes” para representar los números negativos
- El complemento de un número n se obtiene restando n al número más grande que puede representarse.
 - Si el número más grande representable es $b^k - 1$ entonces $-n$ se codifica en complemento a 1 como:

$$C_1^n = b^k - 1 - n$$

- Tiene igual rango que la representación Signo y Magnitud.

■ Problemas de esta representación:

- Existen dos ceros (11...11 y 00...00).
- La implementación de la resta $a - b$ no es simplemente sumar el $a + (-b)$.

-3 en 4 bits:

$$C_1^3 = 2^4 - 1 - 3 =$$

$$C_1^3 = 10000 - 00001 - 00011$$

$$C_1^3 = 01100 = 1100$$

Otra posibilidad:

$$-x = \bar{x}$$

Representación Complemento a 2 ^(1/3)

- El complemento a 2 de un número n se obtiene restando n al número b^k .

$$C_2^n = b^k - n$$

- Permite representar las magnitudes $[-b^{k-1}, b^{k-1} - 1]$
- También se puede calcular sumando uno a la representación en complemento a uno de dicho numero.

$$C_2^n = C_1^n + 1$$

- **Problemas de esta representación:**

- Rango desbalanceado: Se pueden representar un numero negativo más que la cantidad de números positivos.

-3 en 4 bits:

$$C_2^3 = 2^4 - 3 =$$

$$C_2^3 = 10000 - 00011$$

$$C_2^3 = 01101 = 1101$$

Representación Complemento a 2 ^(2/3)

- Los números positivos son iguales que en binario.
- Tiene un sólo cero.
- Todos los números negativos tienen un 1 en el bit mas significativo: por eso se llama el bit de signo.
 - Para saber si un número es negativo, el hardware sólo tiene que testear ese bit.
- Las operaciones aritméticas son sencillas, en particular la resta es un caso particular de la suma.
- Rango: -2^{k-1} a $+2^{k-1} - 1$: *desbalanceado*

Es la representación de los números enteros con signo en todas las computadoras modernas.

Representación Complemento a 2 ^(3/3)

$$n + \bar{n} = 11 \dots 111 = -1$$

$$n + \bar{n} + 1 = 0$$

$$\bar{n} + 1 = -n$$

- Método rápido para calcular el complemento a 2.

Ejemplo:

$$\begin{aligned} -3 &= \bar{3} + 1 = \overline{0011} + 0001 = 1100 + 0001 = \\ &= 1101 = -3 \end{aligned}$$

Magnitudes Signadas: Comparación

	Binario Sin-signo	Complemento a 2	Complemento a 1	Signo y Magnitud
000	0	0	0	0
001	1	1	1	1
010	2	2	2	2
011	3	3	3	3
100	4	-4	-3	0
101	5	-3	-2	-1
110	6	-2	-1	-2
111	7	-1	0	-3

Suma con Signo (1/2)

■ Hay cuatro casos

1. **Ambos números son positivos:**

$$\begin{array}{r} 00000111 \\ + 00000100 \\ \hline 00001011 \end{array} \quad \begin{array}{r} 7 \\ + 4 \\ \hline 11 \end{array}$$

La suma es positiva y, por tanto, es un número binario real (no complementado).

2. **Ambos números son negativos:**

$$\begin{array}{r} 11111011 \\ + 11110111 \\ \hline 111110010 \end{array} \quad \begin{array}{r} -5 \\ + -9 \\ \hline -14 \end{array}$$

Descartar acarreo $\longrightarrow$ 1

El bit de acarreo final no se tiene en cuenta.

La suma es negativa y, por tanto, está en complemento a 2.

3. **El número positivo es mayor que el número negativo en valor absoluto:**

$$\begin{array}{r} 00001111 \\ + 11111010 \\ \hline 100001001 \end{array} \quad \begin{array}{r} 15 \\ + -6 \\ \hline 9 \end{array}$$

Descartar acarreo $\longrightarrow$ 1

El bit de acarreo final no se tiene en cuenta. La suma es positiva y, por tanto, es un número binario real (no complementado).

Suma con Signo (2/2)

4. El número negativo es mayor que el número positivo en valor absoluto:

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \quad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

La suma es negativa y, por tanto, está en complemento a 2.

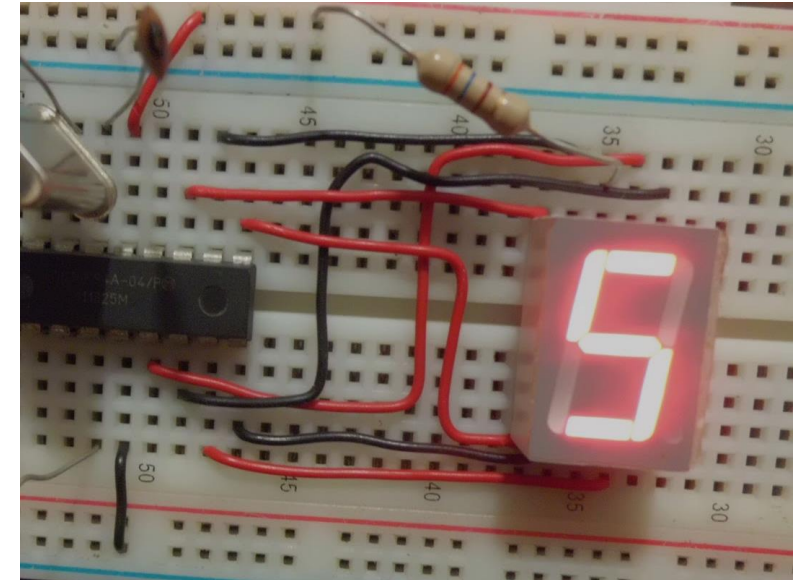
- *En resumen: se utiliza la suma binaria, y siempre se descarta el acarreo.*
- El overflow se produce si el resultado está fuera de rango
 - Sumar operandos con diferente signo no da overflow
 - Sumar dos operandos positivos
 - Hay overflow si el signo del resultado es 1
 - Sumar dos operandos negativos
 - Hay overflow si el signo del resultado es 0

Resta con Signo

- Sumar la negación del segundo operando
- Ejemplo: $7 - 6 = 7 + (-6)$
+7: 0000 0111
- 6: 1111 1010
+1: 0000 0001
- Overflow si el resultado está fuera de rango
 - Restar dos operandos de igual signo no da overflow
 - Restar operando positivo de negativo
 - Overflow si el signo del resultado es 0
 - Restar operando negativo de positivo
 - Overflow si el signo del resultado es 1

Números de precisión fija

- Cuando se opera matemáticamente con números no existe una restricción de tamaño ni para los operadores ni para el resultado.
 - Pero cuando debemos implementar un hardware que los almacene y opere debemos acotar estos números.
 - Existe entonces una solución de compromiso: Los números de precisión fija.
- Los números de precisión fija están **definidos por su largo**, es decir por su cantidad de bits k
 - Además se pueden caracterizar por su forma de representación (sin signo, signo y magnitud, complemento a 2, etc.)



Display 7 segmentos:
Permite visualizar los números del 0 al 9
en el sistema decimal o
del 0 a F para el sistema hexadecimal

Flags ^(1/4)

- En operaciones aritméticas se utilizan señales de 1 bit denominadas comúnmente **Flags**.
 - Estas son muy útiles para caracterizar el resultado de las operaciones.
 - Las 4 principales son: **Zero**, **Carry**, **oVerflow**, y **Negative**.
 - Son independientes entre ellas.
- **Zero (Z)** - Cero
 - Se pone en alto si el resultado de la **última** operación aritmética **es igual a cero**.
 - Se puede utilizar como comparador si se hace la resta de dos números.

Flags (2/4)

■ Carry (C) - Acarreo

- Se pone en alto cuando en la suma existe acarreo (carry) o cuando en la resta existe acarreo negativo (borrow).

- Reglas:

- Es 1 si en la suma se presenta un acarreo en el bit más significativo.

$$1111 + 0001 = 0000 ; \text{carry} = 1$$

- Es 1 si en la resta se presenta un acarreo negativo en el bit más significativo.

$$0000 - 0001 = 1111 ; \text{carry} = 1$$

- En cualquier otro caso es 0.

$$0111 + 0001 = 1000 ; \text{carry} = 0$$

$$1000 - 0001 = 0111 ; \text{carry} = 0$$

Flags ^(3/4)

■ Overflow (V) - Desbordamiento

- Se pone en alto cuando el resultado de la operación no es representable en la precisión elegida.

- Reglas:

- Es 1 si la suma de dos números positivos da un resultado negativo.

$$0100 + 0100 = 1000 ; \textit{overflow} = 1$$

- Es 1 si la suma de dos números negativos da un resultado positivo.

$$1000 + 1000 = 0000 ; \textit{overflow} = 1$$

- En cualquier otro caso es 0.

$$0100 + 0001 = 0101 ; \textit{overflow} = 0$$

$$0110 + 1001 = 1111 ; \textit{overflow} = 0$$

$$0100 + 0001 = 1001 ; \textit{overflow} = 0$$

$$1100 + 1100 = 1000 ; \textit{overflow} = 0$$

■ Negative (N) - Negativo

- Se pone en alto cuando el resultado de la operación es negativo.

Flags ^(4/4)

- **Diferencia entre Carry y Overflow**

- **Carry (C)** – Es muy importante en sistemas binarios SIN signo

$$\begin{array}{r} (7): 111 \\ + (1): 001 \\ \hline (8): 1\ 000 \end{array}$$

- **Overflow (V)** – Es muy importante en sistemas binarios CON signo

$$\begin{array}{r} (+3): 011 \\ + (+1): 001 \\ \hline (+4): 100 \end{array} \text{ (en Complemento a 2 es -4 debido a que con 3 bits no se puede representar el +4 en dicho Sistema de numeración)}$$